

# Programming Assignment 01 B: Loading and Analyzing a JSON File

**Name:** Devi Aditya Dantuluri

Load `myData.json`, convert it to Python dictionaries, display key-value pairs with Pandas, group the documents by country, and count documents per country.

```
In [1]: # Import the json module (parses JSON text into Python objects) and pandas (tabular display/analysis).
import json
import pandas as pd
pd.set_option("display.max_rows", 100) # show all rows of the final table

# Open and load the JSON file. The file starts with a UTF-8 byte-order mark (BOM), so the
# "utf-8-sig" encoding is used; plain "utf-8" raises "Unexpected UTF-8 BOM" on this file.
with open("myData.json", "r", encoding="utf-8-sig") as f:
    data = json.load(f)

# Observe the structure: the top-level object is a Python list, and each element is a dictionary (one document)
print("Top-level type :", type(data).__name__)
print("Number of documents:", len(data))
print("Type of one element:", type(data[0]).__name__)
```

Top-level type : list  
Number of documents: 467  
Type of one element: dict

## Structure of the JSON file

- The file is a JSON **array** ( `[ ... ]` ), which `json.load` converts to a Python **list**.
- Each element of the array is a JSON **object** ( `{ ... }` ), converted to a Python **dict**. **Each dictionary is one document** (one place record). The documents are identified by their position in the list (index 0, 1, 2, ...), and each also carries a unique `woeid` (Where-On-Earth ID) and a `name`.
- Every document has the same 7 keys: `name`, `placeType`, `url`, `parentid`, `country`, `woeid`, `countryCode`.
- `placeType` is itself a **nested object** with two keys, `code` and `name`, so it becomes a nested dictionary.
- The first document ( "Worldwide" ) has an empty string for `country` and `null` (Python `None`) for `countryCode`; that is the document with no country information.

```
In [2]: # Look at the first three documents to see the key-value structure (including the nested placeType dictionary)
for i in range(3):
    print(f"Document {i}:")
    print(json.dumps(data[i], indent=4))
```

```

Document 0:
{
  "name": "Worldwide",
  "placeType": {
    "code": 19,
    "name": "Supername"
  },
  "url": "http://where.yahooapis.com/v1/place/1",
  "parentid": 0,
  "country": "",
  "woeid": 1,
  "countryCode": null
}
Document 1:
{
  "name": "Winnipeg",
  "placeType": {
    "code": 7,
    "name": "Town"
  },
  "url": "http://where.yahooapis.com/v1/place/2972",
  "parentid": 23424775,
  "country": "Canada",
  "woeid": 2972,
  "countryCode": "CA"
}
Document 2:
{
  "name": "Ottawa",
  "placeType": {
    "code": 7,
    "name": "Town"
  },
  "url": "http://where.yahooapis.com/v1/place/3369",
  "parentid": 23424775,
  "country": "Canada",
  "woeid": 3369,
  "countryCode": "CA"
}

```

```

In [3]: # Identify the key-value pairs of a single document. Example: "name": "Winnipeg" -> key = "name", value = "Winnipeg"
doc = data[1] # the Winnipeg document
print("Keys in each document:", list(doc.keys()))
print()
for key, value in doc.items():
    print(f"key = {key!r:15} value = {value!r}")

```

Keys in each document: ['name', 'placeType', 'url', 'parentid', 'country', 'woeid', 'countryCode']

|                     |                                                    |
|---------------------|----------------------------------------------------|
| key = 'name'        | value = 'Winnipeg'                                 |
| key = 'placeType'   | value = {'code': 7, 'name': 'Town'}                |
| key = 'url'         | value = 'http://where.yahooapis.com/v1/place/2972' |
| key = 'parentid'    | value = 23424775                                   |
| key = 'country'     | value = 'Canada'                                   |
| key = 'woeid'       | value = 2972                                       |
| key = 'countryCode' | value = 'CA'                                       |

## Key-value pairs in two tabular columns

```

In [4]: # Build a two-column DataFrame (Key, Value) from the key-value pairs of the Winnipeg document.
kv_df = pd.DataFrame(list(doc.items()), columns=["Key", "Value"])
kv_df

```

Out[4]:

|   | Key         | Value                                    |
|---|-------------|------------------------------------------|
| 0 | name        | Winnipeg                                 |
| 1 | placeType   | {'code': 7, 'name': 'Town'}              |
| 2 | url         | http://where.yahooapis.com/v1/place/2972 |
| 3 | parentid    | 23424775                                 |
| 4 | country     | Canada                                   |
| 5 | woeid       | 2972                                     |
| 6 | countryCode | CA                                       |

```
In [5]: # Same two-column view for every document, with a "Document" column so it is clear which document each pair
rows = []
for idx, d in enumerate(data):
    for key, value in d.items():
        rows.append({"Document": f"{idx}: {d['name']}", "Key": key, "Value": value})
all_kv_df = pd.DataFrame(rows)
print("Total key-value pairs:", len(all_kv_df))
all_kv_df.head(21) # first three documents (7 keys each)
```

Total key-value pairs: 3269

Out[5]:

|    | Document     | Key         | Value                                    |
|----|--------------|-------------|------------------------------------------|
| 0  | 0: Worldwide | name        | Worldwide                                |
| 1  | 0: Worldwide | placeType   | {'code': 19, 'name': 'Supernature'}      |
| 2  | 0: Worldwide | url         | http://where.yahooapis.com/v1/place/1    |
| 3  | 0: Worldwide | parentid    | 0                                        |
| 4  | 0: Worldwide | country     |                                          |
| 5  | 0: Worldwide | woeid       | 1                                        |
| 6  | 0: Worldwide | countryCode | None                                     |
| 7  | 1: Winnipeg  | name        | Winnipeg                                 |
| 8  | 1: Winnipeg  | placeType   | {'code': 7, 'name': 'Town'}              |
| 9  | 1: Winnipeg  | url         | http://where.yahooapis.com/v1/place/2972 |
| 10 | 1: Winnipeg  | parentid    | 23424775                                 |
| 11 | 1: Winnipeg  | country     | Canada                                   |
| 12 | 1: Winnipeg  | woeid       | 2972                                     |
| 13 | 1: Winnipeg  | countryCode | CA                                       |
| 14 | 2: Ottawa    | name        | Ottawa                                   |
| 15 | 2: Ottawa    | placeType   | {'code': 7, 'name': 'Town'}              |
| 16 | 2: Ottawa    | url         | http://where.yahooapis.com/v1/place/3369 |
| 17 | 2: Ottawa    | parentid    | 23424775                                 |
| 18 | 2: Ottawa    | country     | Canada                                   |
| 19 | 2: Ottawa    | woeid       | 3369                                     |
| 20 | 2: Ottawa    | countryCode | CA                                       |

```
In [6]: # Flatten the list of dictionaries into a regular table: one row per document, one column per key.
# json_normalize expands the nested placeType dictionary into placeType.code and placeType.name.
df = pd.json_normalize(data)
print(df.shape)
df.head(10)
```

(467, 8)

|   | name       | url                                       | parentid | country        | woeid | countryCode | placeType.code |
|---|------------|-------------------------------------------|----------|----------------|-------|-------------|----------------|
| 0 | Worldwide  | http://where.yahooapis.com/v1/place/1     | 0        |                | 1     | NaN         | 1              |
| 1 | Winnipeg   | http://where.yahooapis.com/v1/place/2972  | 23424775 | Canada         | 2972  | CA          |                |
| 2 | Ottawa     | http://where.yahooapis.com/v1/place/3369  | 23424775 | Canada         | 3369  | CA          |                |
| 3 | Quebec     | http://where.yahooapis.com/v1/place/3444  | 23424775 | Canada         | 3444  | CA          |                |
| 4 | Montreal   | http://where.yahooapis.com/v1/place/3534  | 23424775 | Canada         | 3534  | CA          |                |
| 5 | Toronto    | http://where.yahooapis.com/v1/place/4118  | 23424775 | Canada         | 4118  | CA          |                |
| 6 | Edmonton   | http://where.yahooapis.com/v1/place/8676  | 23424775 | Canada         | 8676  | CA          |                |
| 7 | Calgary    | http://where.yahooapis.com/v1/place/8775  | 23424775 | Canada         | 8775  | CA          |                |
| 8 | Vancouver  | http://where.yahooapis.com/v1/place/9807  | 23424775 | Canada         | 9807  | CA          |                |
| 9 | Birmingham | http://where.yahooapis.com/v1/place/12723 | 23424975 | United Kingdom | 12723 | GB          |                |

## Group the documents by the key `country`

```
In [7]: # Replace the empty country string with an explicit label so the document without country information is not lost
df["country"] = df["country"].replace("", "No country (Worldwide)")

# groupby("country") partitions the documents by their country value.
grouped = df.groupby("country")

# List the documents (place names) that belong to each country group.
for country, group in grouped:
    print(f"{country} ({len(group)} documents): {' '.join(group['name'].tolist())}")
```

Algeria (2 documents): Algiers, Algeria  
Argentina (5 documents): Mendoza, Córdoba, Rosario, Buenos Aires, Argentina  
Australia (8 documents): Perth, Adelaide, Brisbane, Canberra, Darwin, Melbourne, Sydney, Australia  
Austria (2 documents): Vienna, Austria  
Bahrain (1 documents): Bahrain  
Belarus (5 documents): Brest, Grodno, Gomel, Minsk, Belarus  
Belgium (1 documents): Belgium  
Brazil (16 documents): Brasília, Belém, Belo Horizonte, Curitiba, Porto Alegre, Recife, Rio de Janeiro, Salvador, São Paulo, Campinas, Fortaleza, Goiânia, Manaus, São Luís, Guarulhos, Brazil  
Canada (9 documents): Winnipeg, Ottawa, Quebec, Montreal, Toronto, Edmonton, Calgary, Vancouver, Canada  
Chile (4 documents): Santiago, Concepcion, Valparaiso, Chile  
Colombia (5 documents): Bogotá, Cali, Medellín, Barranquilla, Colombia  
Denmark (1 documents): Denmark  
Dominican Republic (2 documents): Santo Domingo, Dominican Republic  
Ecuador (3 documents): Quito, Guayaquil, Ecuador  
Egypt (4 documents): Giza, Cairo, Alexandria, Egypt  
France (11 documents): Bordeaux, Lille, Lyon, Marseille, Montpellier, Nantes, Paris, Rennes, Strasbourg, Toulouse, France  
Germany (13 documents): Berlin, Bremen, Dortmund, Dresden, Dusseldorf, Essen, Frankfurt, Hamburg, Cologne, Leipzig, Munich, Stuttgart, Germany  
Ghana (3 documents): Accra, Kumasi, Ghana  
Greece (3 documents): Athens, Thessaloniki, Greece  
Guatemala (2 documents): Guatemala City, Guatemala  
India (22 documents): Nagpur, Lucknow, Kanpur, Patna, Ranchi, Kolkata, Srinagar, Amritsar, Jaipur, Ahmedabad, Rajkot, Surat, Bhopal, Indore, Thane, Mumbai, Pune, Hyderabad, Bangalore, Chennai, Delhi, India  
Indonesia (12 documents): Bekasi, Depok, Pekanbaru, Surabaya, Makassar, Bandung, Jakarta, Medan, Palembang, Semarang, Tangerang, Indonesia  
Ireland (4 documents): Cork, Dublin, Galway, Ireland  
Israel (4 documents): Haifa, Tel Aviv, Jerusalem, Israel  
Italy (8 documents): Bologna, Genoa, Milan, Naples, Palermo, Rome, Turin, Italy  
Japan (22 documents): Kitakyushu, Saitama, Chiba, Fukuoka, Hamamatsu, Hiroshima, Kawasaki, Kobe, Kumamoto, Nagoya, Niigata, Sagami-hara, Sapporo, Sendai, Takamatsu, Tokyo, Yokohama, Okinawa, Osaka, Kyoto, Japan, Okayama  
Jordan (2 documents): Amman, Jordan  
Kenya (3 documents): Mombasa, Nairobi, Kenya  
Korea (15 documents): Goyang, Yongin, Ansan, Bucheon, Busan, Changwon, Daegu, Gwangju, Incheon, Seongnam, Suwon

won, Ulsan, Seoul, Daejeon, Korea  
 Kuwait (1 documents): Kuwait  
 Latvia (2 documents): Riga, Latvia  
 Lebanon (1 documents): Lebanon  
 Malaysia (8 documents): Kajang, Ipoh, Johor Bahru, Klang, Kuala Lumpur, Malaysia, Petaling, Hulu Langat  
 Mexico (24 documents): Acapulco, Aguascalientes, Chihuahua, Mexico City, Ciudad Juarez, Nezahualcóyotl, Culiacán, Ecatepec de Morelos, Guadalajara, Hermosillo, León, Mérida, Mexicali, Monterrey, Morelia, Naucalpan de Juárez, Puebla, Querétaro, Saltillo, San Luis Potosí, Tijuana, Toluca, Zapopan, Mexico  
 Netherlands (5 documents): Den Haag, Amsterdam, Rotterdam, Utrecht, Netherlands  
 New Zealand (2 documents): Auckland, New Zealand  
 Nigeria (7 documents): Benin City, Ibadan, Kaduna, Kano, Lagos, Port Harcourt, Nigeria  
 No country (Worldwide) (1 documents): Worldwide  
 Norway (3 documents): Bergen, Oslo, Norway  
 Oman (2 documents): Muscat, Oman  
 Pakistan (6 documents): Karachi, Lahore, Multan, Rawalpindi, Faisalabad, Pakistan  
 Panama (1 documents): Panama  
 Peru (2 documents): Lima, Peru  
 Philippines (12 documents): Calocan, Makati, Pasig, Taguig, Antipolo, Cagayan de Oro, Cebu City, Davao City, Manila, Quezon City, Zamboanga City, Philippines  
 Poland (7 documents): Gdańsk, Kraków, Łódź, Poznań, Warsaw, Wrocław, Poland  
 Portugal (1 documents): Portugal  
 Puerto Rico (1 documents): Puerto Rico  
 Qatar (1 documents): Qatar  
 Russia (20 documents): Chelyabinsk, Khabarovsk, Krasnodar, Krasnoyarsk, Samara, Voronezh, Yekaterinburg, Irkutsk, Kazan, Moscow, Nizhny Novgorod, Novosibirsk, Omsk, Perm, Rostov-on-Don, Saint Petersburg, Ufa, Vladivostok, Volgograd, Russia  
 Saudi Arabia (7 documents): Medina, Dammam, Riyadh, Jeddah, Mecca, Saudi Arabia, Ahhsa  
 Singapore (2 documents): Singapore, Singapore  
 South Africa (7 documents): Durban, Johannesburg, Port Elizabeth, Pretoria, Soweto, Cape Town, South Africa  
 Spain (11 documents): Barcelona, Bilbao, Las Palmas, Madrid, Malaga, Murcia, Palma, Seville, Valencia, Zaragoza, Spain  
 Sweden (3 documents): Gothenburg, Stockholm, Sweden  
 Switzerland (4 documents): Geneva, Lausanne, Zurich, Switzerland  
 Thailand (2 documents): Bangkok, Thailand  
 Turkey (13 documents): Mersin, Adana, Ankara, Antalya, Bursa, Diyarbakır, Eskişehir, Gaziantep, Istanbul, Izmir, Kayseri, Konya, Turkey  
 Ukraine (8 documents): Dnipropetrovsk, Donetsk, Kharkiv, Kyiv, Lviv, Odesa, Zaporozhye, Ukraine  
 United Arab Emirates (4 documents): Sharjah, Abu Dhabi, Dubai, United Arab Emirates  
 United Kingdom (27 documents): Birmingham, Blackpool, Bournemouth, Brighton, Bristol, Cardiff, Coventry, Derby, Edinburgh, Glasgow, Hull, Leeds, Leicester, Liverpool, Manchester, Middlesbrough, Newcastle, Nottingham, Plymouth, Portsmouth, Preston, Sheffield, Stoke-on-Trent, Swansea, London, Belfast, United Kingdom  
 United States (64 documents): Albuquerque, Atlanta, Austin, Baltimore, Baton Rouge, Birmingham, Boston, Charlotte, Chicago, Cincinnati, Cleveland, Colorado Springs, Columbus, Dallas-Ft. Worth, Denver, Detroit, El Paso, Fresno, Greensboro, Harrisburg, Honolulu, Houston, Indianapolis, Jackson, Jacksonville, Kansas City, Las Vegas, Long Beach, Los Angeles, Louisville, Memphis, Mesa, Miami, Milwaukee, Minneapolis, Nashville, New Haven, New Orleans, New York, Norfolk, Oklahoma City, Omaha, Orlando, Philadelphia, Phoenix, Pittsburgh, Portland, Providence, Raleigh, Richmond, Sacramento, St. Louis, Salt Lake City, San Antonio, San Diego, San Francisco, San Jose, Seattle, Tallahassee, Tampa, Tucson, Virginia Beach, Washington, United States  
 Venezuela (10 documents): Caracas, Maracaibo, Maracay, Valencia, Barcelona, Ciudad Guayana, Turmero, Barquisimeto, Maturín, Venezuela  
 Vietnam (6 documents): Hanoi, Hai Phong, Can Tho, Da Nang, Ho Chi Minh City, Vietnam

## Number of unique countries (including the one with no country information)

```
In [8]: # nunique() counts the distinct values of the country column; the "No country" label is included as one value
n_unique = df["country"].nunique()
print("Unique countries (including the no-country entry):", n_unique)
print("Unique countries (excluding the no-country entry):", n_unique - 1)
print()
print(sorted(df["country"].unique()))
```

Unique countries (including the no-country entry): 63

Unique countries (excluding the no-country entry): 62

```
['Algeria', 'Argentina', 'Australia', 'Austria', 'Bahrain', 'Belarus', 'Belgium', 'Brazil', 'Canada', 'Chile', 'Colombia', 'Denmark', 'Dominican Republic', 'Ecuador', 'Egypt', 'France', 'Germany', 'Ghana', 'Greece', 'Guatemala', 'India', 'Indonesia', 'Ireland', 'Israel', 'Italy', 'Japan', 'Jordan', 'Kenya', 'Korea', 'Kuwait', 'Latvia', 'Lebanon', 'Malaysia', 'Mexico', 'Netherlands', 'New Zealand', 'Nigeria', 'No country (Worldwide)', 'Norway', 'Oman', 'Pakistan', 'Panama', 'Peru', 'Philippines', 'Poland', 'Portugal', 'Puerto Rico', 'Qatar', 'Russia', 'Saudi Arabia', 'Singapore', 'South Africa', 'Spain', 'Sweden', 'Switzerland', 'Thailand', 'Turkey', 'Ukraine', 'United Arab Emirates', 'United Kingdom', 'United States', 'Venezuela', 'Vietnam']
```

## Documents per country: DataFrame with Country and DocumentCount

```
In [9]: # groupby + count() gives the number of documents in each country group; rename the columns as requested.
country_counts = (
    df.groupby("country")["name"].count()
    .reset_index()
    .rename(columns={"country": "Country", "name": "DocumentCount"})
    .sort_values("DocumentCount", ascending=False)
    .reset_index(drop=True)
)
country_counts
```

Out[9]:

|  | Country | DocumentCount |
|--|---------|---------------|
|--|---------|---------------|

|    |                |    |
|----|----------------|----|
| 0  | United States  | 64 |
| 1  | United Kingdom | 27 |
| 2  | Mexico         | 24 |
| 3  | Japan          | 22 |
| 4  | India          | 22 |
| 5  | Russia         | 20 |
| 6  | Brazil         | 16 |
| 7  | Korea          | 15 |
| 8  | Germany        | 13 |
| 9  | Turkey         | 13 |
| 10 | Philippines    | 12 |
| 11 | Indonesia      | 12 |
| 12 | Spain          | 11 |
| 13 | France         | 11 |
| 14 | Venezuela      | 10 |
| 15 | Canada         | 9  |
| 16 | Italy          | 8  |
| 17 | Australia      | 8  |
| 18 | Malaysia       | 8  |
| 19 | Ukraine        | 8  |
| 20 | South Africa   | 7  |
| 21 | Nigeria        | 7  |
| 22 | Poland         | 7  |
| 23 | Saudi Arabia   | 7  |
| 24 | Vietnam        | 6  |
| 25 | Pakistan       | 6  |
| 26 | Argentina      | 5  |
| 27 | Belarus        | 5  |
| 28 | Colombia       | 5  |
| 29 | Netherlands    | 5  |
| 30 | Egypt          | 4  |
| 31 | Chile          | 4  |

|    |                        |   |
|----|------------------------|---|
| 32 | United Arab Emirates   | 4 |
| 33 | Switzerland            | 4 |
| 34 | Israel                 | 4 |
| 35 | Ireland                | 4 |
| 36 | Ghana                  | 3 |
| 37 | Sweden                 | 3 |
| 38 | Norway                 | 3 |
| 39 | Kenya                  | 3 |
| 40 | Ecuador                | 3 |
| 41 | Greece                 | 3 |
| 42 | Algeria                | 2 |
| 43 | Austria                | 2 |
| 44 | Latvia                 | 2 |
| 45 | Jordan                 | 2 |
| 46 | Guatemala              | 2 |
| 47 | Thailand               | 2 |
| 48 | Singapore              | 2 |
| 49 | Oman                   | 2 |
| 50 | Peru                   | 2 |
| 51 | Dominican Republic     | 2 |
| 52 | New Zealand            | 2 |
| 53 | Belgium                | 1 |
| 54 | Denmark                | 1 |
| 55 | Kuwait                 | 1 |
| 56 | Bahrain                | 1 |
| 57 | Puerto Rico            | 1 |
| 58 | No country (Worldwide) | 1 |
| 59 | Lebanon                | 1 |
| 60 | Portugal               | 1 |
| 61 | Panama                 | 1 |
| 62 | Qatar                  | 1 |

```
In [10]: # Sanity check: the counts must add up to the total number of documents in the file.
print("Sum of DocumentCount:", country_counts["DocumentCount"].sum())
print("Total documents      :", len(data))
```

```
Sum of DocumentCount: 467
Total documents      : 467
```